

Node.js + fs Module: Complete CRUD Notes

Complete notes based on the lesson, preserving all concepts, explanations, examples, code, mental models, roadmap, and mini-project guidance.

Prerequisite: HTML, CSS, and basic JavaScript through DOM/event handling.

1. Where fs Comes In

You've probably done:

```
document.getElementById("btn").addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

That JavaScript runs in the browser.

The browser JavaScript environment gives you things like:

```
document  
window  
localStorage  
DOM  
events  
fetch()
```

Node.js gives you access to the computer/server environment:

```
Files  
Folders  
Operating system  
HTTP server  
Databases  
Environment variables  
Processes
```

And one of the most important Node.js modules for working with files is `fs`.

`fs` = File System

Conceptually:

```
HTML + CSS + JS  
↓  
Browser  
↓  
DOM
```

```
JavaScript  
↓  
Node.js  
↓  
fs  
↓  
Files & Folders
```

2. First: Install Node.js

Check whether Node.js is already installed. Open the terminal:

```
node -v
```

You should get something like `v22.x.x`.

```
npm -v
```

If both work, you're ready.

3. Create Your First Node Project

Make a folder named `fs-learning` and open it in VS Code.

```
npm init -y
```

You'll get `package.json`.

Create `app.js`:

```
fs-learning/
├──
├── package.json
└── app.js
```

Inside `app.js`:

```
console.log("Hello Node");
```

Run:

```
node app.js
```

Output:

```
Hello Node
```

Congratulations — you're running JavaScript outside the browser.

4. Importing the fs Module

Node provides `fs` as a built-in module. You don't need to install it.

Two common styles:

CommonJS

```
const fs = require("fs");
```

ES Modules

```
import fs from "fs";
```

For initial learning, use:

```
const fs = require("fs");
```

It's straightforward.

5. CRUD — The Big Picture

CRUD means:

Operation	Meaning	File equivalent
C	Create	Create a file
R	Read	Read a file
U	Update	Modify a file
D	Delete	Delete a file

The goal is:

```
Create
  ↓
Read
```

↓
Update
↓
Delete

Core fs methods:

```
fs.writeFile()  
fs.readFile()  
fs.appendFile()  
fs.unlink()
```

Synchronous versions:

```
fs.writeFileSync()  
fs.readFileSync()  
fs.appendFileSync()  
fs.unlinkSync()
```

We'll learn both, but async should become your normal habit.

6. CREATE — Creating a File

Let's create `data.txt`.

```
const fs = require("fs");  
  
fs.writeFile("data.txt", "Hello World", (err) => {  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
  console.log("File created successfully");  
});
```

Run `node app.js`. You'll see `File created successfully`.

Your folder becomes:

```
fs-learning/  
■  
■■■ app.js  
■■■ data.txt  
■■■ package.json
```

`data.txt` contains:

```
Hello World
```

7. Understanding writeFile()

```
fs.writeFile("data.txt", "Hello World", callback);
```

It has three important parts:

```
filename  
↓  
"data.txt"  
  
content  
↓  
"Hello World"  
  
callback  
↓  
(err) => {}
```

General structure:

```
fs.writeFile(file, data, callback);
```

8. What Is err?

Node's filesystem APIs commonly use:

```
(err) => {}
```

If something goes wrong, `err` contains information about the error. If everything succeeds, `err === null`.

```
fs.writeFile("data.txt", "Hello", (err) => {  
  
  if (err) {  
    console.log("Something went wrong");  
    return;  
  }  
  
  console.log("Success");  
});
```

Think:

```
Did operation fail?  
  ↓  
  err  
 /   \  
yes   no  
↓     ↓  
error success
```

9. READ — Reading a File

```
const fs = require("fs");  
  
fs.readFile("data.txt", "utf8", (err, data) => {  
  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
  console.log(data);  
});
```

Output: Hello World.

Notice `utf8`. Files are fundamentally stored as bytes. Without encoding:

```
fs.readFile("data.txt", (err, data) => {  
  console.log(data);  
});
```

you'll typically get a `Buffer`. With `utf8`, Node gives you a normal string.

Generally use:

```
fs.readFile("data.txt", "utf8", ...)
```

10. UPDATE — Modifying a File

There are two common interpretations of update.

Replace the Entire Content

Use `writeFile()`.

```
fs.writeFile("data.txt", "New content", (err) => {  
  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
});
```

```
    console.log("File updated");
  });
```

Old content: Hello World

New content: New content

Important:

```
writeFile()
  ↓
If file exists → overwrite it
If file doesn't exist → create it
```

11. APPEND — Adding Content

Suppose the file contains Hello World and you want Hello World followed by Welcome to Node.js.

```
fs.appendFile("data.txt", "\nWelcome to Node.js", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("Data added");
});
```

Now:

```
Hello World
Welcome to Node.js
```

Difference:

```
writeFile()
  ↓
Replace content.

appendFile()
  ↓
Add content at the end.
```

12. DELETE — Deleting a File

```
fs.unlink("data.txt", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("File deleted");
});
```

The file disappears.

Basic CRUD toolkit:

```
// CREATE / REPLACE
fs.writeFile()

// READ
fs.readFile()

// UPDATE / ADD
fs.appendFile()

// DELETE
fs.unlink()
```

13. Synchronous Versions

You can use synchronous methods such as `fs.writeFileSync()`.

```
const fs = require("fs");

fs.writeFileSync("data.txt", "Hello");

console.log("Done");
```

This is easier to understand initially.

14. Sync vs Asynchronous

Synchronous

```
fs.writeFileSync("data.txt", "Hello");

console.log("Done");
```

Node waits: write file → wait → finished → console.log.

Asynchronous

```
fs.writeFile("data.txt", "Hello", () => {
  console.log("File finished");
});

console.log("Done");
```

Potential output:

```
Done
File finished
```

Node doesn't sit around waiting for the file operation.

15. Why Node.js Likes Asynchronous Operations

Imagine a server receiving 1000 users. Suppose every request requires reading a file.

With blocking code:

```
User 1 → read file → WAIT
                        ↓
                    finished
                        ↓
User 2 → read file
```

That's bad for scalability.

With asynchronous code:

```
User 1 → request file ██████████→ result
User 2 → request file ████████→ result
User 3 → request file ██████████→ result
User 4 → request file ██████████→ result
```

Node can continue doing other work while the filesystem operation happens.

You don't need to master the event loop yet, but understanding async operations is going to become very important.

16. CRUD with JSON — Where It Gets Interesting

Working with `.txt` files teaches the basics. Real applications often store structured data.

```
[
  {
```

```

    "id": 1,
    "name": "Shan",
    "age": 21
  },
  {
    "id": 2,
    "name": "Ali",
    "age": 22
  }
]

```

Create `users.json` with that structure.

```

const fs = require("fs");

const users = [
  {
    id: 1,
    name: "Shan",
    age: 21
  },
  {
    id: 2,
    name: "Ali",
    age: 22
  }
];

fs.writeFile(
  "users.json",
  JSON.stringify(users, null, 2),
  (err) => {

    if (err) {
      console.log(err);
      return;
    }

    console.log("Users saved");
  }
);

```

17. Why `JSON.stringify()`?

A JavaScript object:

```

const user = {
  name: "Shan",
  age: 21
};

```

is an actual JavaScript object. A JSON file needs text:

```

{
  "name": "Shan",
  "age": 21
}

```

So:

```
JSON.stringify(user)
```

converts JavaScript object → string → JSON.

And `JSON.parse()` does the reverse:

```

JSON string
  ↓
JSON.parse()
  ↓
JavaScript object

```

These two are absolutely essential.

18. Reading JSON Data

```
const fs = require("fs");

fs.readFile("users.json", "utf8", (err, data) => {

  if (err) {
    console.log(err);
    return;
  }

  const users = JSON.parse(data);

  console.log(users);
});
```

Now users is a normal JavaScript array.

```
console.log(users[0].name);
```

Output: Shan.

19. Real CRUD with JSON

Imagine our database is `users.json`. We want:

```
CREATE USER
READ USERS
UPDATE USER
DELETE USER
```

CREATE

```
fs.readFile("users.json", "utf8", (err, data) => {

  if (err) {
    console.log(err);
    return;
  }

  const users = JSON.parse(data);

  const newUser = {
    id: 3,
    name: "Ahmed",
    age: 20
  };

  users.push(newUser);

  fs.writeFile(
    "users.json",
    JSON.stringify(users, null, 2),
    (err) => {

      if (err) {
        console.log(err);
        return;
      }

      console.log("User added");
    }
  );
});
```

Important pattern:

```
read file
↓
JSON.parse()
↓
JavaScript array
```

↓
users.push()
↓
JSON.stringify()
↓
write file

READ

```
fs.readFile("users.json", "utf8", (err, data) => {  
  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
  const users = JSON.parse(data);  
  
  console.log(users);  
});
```

You can also search:

```
const user = users.find(user => user.id === 2);  
  
console.log(user);
```

UPDATE

Suppose id = 2 needs a new age.

```
fs.readFile("users.json", "utf8", (err, data) => {  
  
  if (err) {  
    console.log(err);  
    return;  
  }  
  
  const users = JSON.parse(data);  
  
  const user = users.find(user => user.id === 2);  
  
  if (!user) {  
    console.log("User not found");  
    return;  
  }  
  
  user.age = 25;  
  
  fs.writeFile(  
    "users.json",  
    JSON.stringify(users, null, 2),  
    (err) => {  
  
      if (err) {  
        console.log(err);  
        return;  
      }  
  
      console.log("User updated");  
    }  
  );  
});
```

Pattern:

READ
↓
parse
↓
find object
↓
modify object
↓

```
stringify
↓
WRITE
```

DELETE

```
fs.readFile("users.json", "utf8", (err, data) => {

  if (err) {
    console.log(err);
    return;
  }

  let users = JSON.parse(data);

  users = users.filter(user => user.id !== 2);

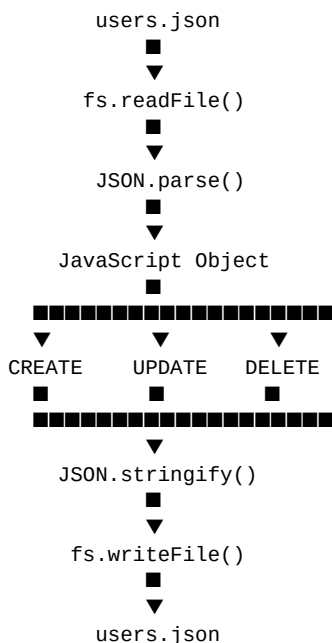
  fs.writeFile(
    "users.json",
    JSON.stringify(users, null, 2),
    (err) => {

      if (err) {
        console.log(err);
        return;
      }

      console.log("User deleted");
    }
  );
});
```

`users.filter(user => user.id !== 2)` creates a new array containing every user except ID 2.

20. Your Mental Model



For JSON-file CRUD, memorize this pipeline. It is basically a tiny homemade database.

21. Important Upgrade: Promises

Eventually callback-heavy code like this becomes annoying:

```
fs.readFile("users.json", "utf8", (err, data) => {
  // ...
});
```

Node provides `fs.promises`.

```
const fs = require("fs").promises;

async function readUsers() {
  try {
    const data = await fs.readFile("users.json", "utf8");
    const users = JSON.parse(data);
    console.log(users);
  } catch (err) {
    console.log(err);
  }
}

readUsers();
```

This is cleaner. It connects directly to the JavaScript concepts you should learn next:

```
Promises
  ↓
async/await
  ↓
Node.js async programming
```

Don't skip Promises and `async/await`.

22. What You Should Learn in fs

Level 1 — Files

```
writeFile()
readFile()
appendFile()
unlink()
```

Level 2 — Synchronous Versions

```
writeFileSync()
readFileSync()
appendFileSync()
unlinkSync()
```

Level 3 — File Information

```
stat()
```

Level 4 — Directories

```
mkdir()
readdir()
rmdir()
```

And:

```
rm()
```

Level 5 — Rename / Move

```
rename()
```

Level 6 — File Existence / Path Handling

Learn the `path` module alongside `fs`.

```
const path = require("path");

const filePath = path.join(__dirname, "data", "users.json");

console.log(filePath);
```

path becomes extremely important when working with real projects.

23. First Mini-Project — Student Record Manager

Don't just watch tutorials. Build a Student Record Manager.

Project structure:

```
student-manager/
■
■ ■ ■ app.js
■ ■ ■ students.json
■ ■ ■ package.json
```

Your program should support:

1. Add student
2. View all students
3. Find student by ID
4. Update student
5. Delete student
6. Exit

Example data:

```
[
  {
    "id": 1,
    "name": "Shan",
    "course": "CSE",
    "marks": 85
  }
]
```

Your Node program should perform:

```
CREATE → add student
READ   → display students
UPDATE → modify marks/name
DELETE → remove student
```

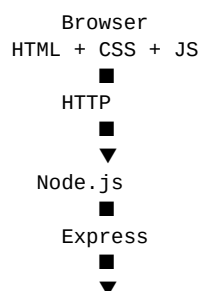
Do not use a database yet. Use:

```
Node.js
+
fs
+
JSON
```

That will force you to understand exactly how CRUD works.

24. Connecting It to Your Existing HTML/JS

Eventually you'll have:



fs / DB
▼
users.json

Your HTML form might be:

Name:
Age:

[Add User]

Browser JavaScript can use `fetch(...)` to send data to Node.

Node receives a request such as:

`POST /users`

Then your backend can perform:

```
fs.readFile()  
JSON.parse()  
users.push()  
JSON.stringify()  
fs.writeFile()
```

Then the backend responds. This is where your current frontend knowledge starts turning into full-stack development.

25. Recommended Learning Order

Part 1: Node.js fundamentals
→ modules, require, npm, package.json

Part 2: fs module
→ files, folders, paths, CRUD

Part 3: Promises + async/await
→ stop relying on callback syntax

Part 4: JSON CRUD project
→ Student Record Manager

Part 5: HTTP module
→ build your first server without Express

Part 6: REST API concepts
→ GET, POST, PUT, PATCH, DELETE

Part 7: Express.js
→ proper backend development

Part 8: Connect your HTML/JS frontend
→ `fetch()` + API

Part 9: Database
→ MongoDB or PostgreSQL

Part 10: Authentication + real full-stack project

If you follow that sequence, you won't just know that `fs` has `readFile` and `writeFile`; you'll understand why it exists, how CRUD works underneath, and how it eventually becomes a real backend API.